Analytics Engineering: Session 8

Practice Session II: Advanced Modeling & Refactoring

Goal

Consolidate Sessions 6-7: Apply materializations, Jinja, macros, and packages to improve our project.

Tasks:

1. Refactor "spaghetti SQL" into modular dbt models
2. Create a custom macro for repeated logic
3. Use package macros to simplify transformations
4. Optimize the DAG for performance
5. Measure and compare build times

Exercise 1: Refactor Spaghetti SQL

Take this monolithic query and break it into modular dbt models:

```sql
-- This is BAD: everything in one query
select o.order_id, c.name, c.email,
    sum(oi.quantity * oi.unit_price) as total,
    s.carrier, s.delivered_at - o.order_date as days
from orders o
join customers c on o.customer_id = c.customer_id
join order_items oi on o.order_id = oi.order_id
join shipping s on o.order_id = s.order_id
group by 1, 2, 3, 5, 6
```

Refactor into: `stg_*` → `int_*` → `mart_*` models using `{{ ref() }}`.

Exercise 2: Create a Custom Macro

Identify repeated logic in your models and extract it into a macro.

Example: A `classify_amount` macro for revenue tiers:

```
-- macros/classify_amount.sql
{% macro classify_amount(column_name) %}
    case
        when {{ column_name }} < 50 then 'low'
        when {{ column_name }} < 200 then 'medium'
        else 'high'
    end
{% endmacro %}
```

Use it across multiple models: `{{ classify_amount('total_amount') }}`

Exercise 3: Use Package Macros

Apply dbt-utils to your project:

Surrogate Keys — Replace composite keys with hash-based keys:

```
{{ dbt_utils.generate_surrogate_key(['order_id', 'product_id']) }}
    as order_item_key
```

dbt-expectations — Add data quality tests:

```
columns:
  - name: total_amount
    tests:
      - dbt_expectations.expect_column_values_to_be_between:
          min_value: 0
```

Exercise 4: Optimize Materializations

Review and optimize materializations across the project:

```yaml
# dbt_project.yml
models:
  dbt_ie:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
```

Measure: Run `dbt build` and compare execution times in the logs.

Exercise 5: Audit the Lineage Graph

Open the lineage graph and check for:

- **Fan-out**: Models with too many direct children (consider intermediate layer).
- **Direct source references in marts**: Should always go through staging.
- **Orphan models**: Models with no downstream consumers.
- **Clean flow**: Sources → Staging → Intermediate → Marts (left to right).

What have we learned in this session

- Refactored monolithic SQL into modular dbt models
- Created custom macros for reusable business logic
- Applied dbt-utils surrogate keys and dbt-expectations tests
- Optimized materializations and measured build times
- Audited the DAG for anti-patterns

Next Session: Advanced Testing, Documentation & Exposures.